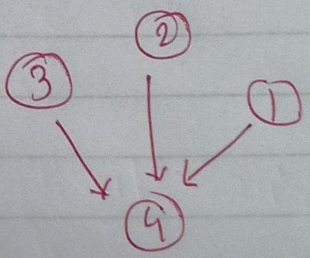
Que-3 The neural network of the virtual world is represented as a connected directed graph data structure, i.e. it has n nodes numbered from 1 to n, and n-1 edges where the i^{th} edge connects node g_from[i] to g_to[i]

select any node as root then reverse as many edges as possible necessary to have all edge flowing down, that is, away from the root. Choose root that required minimum number of changes and report that number of changes.

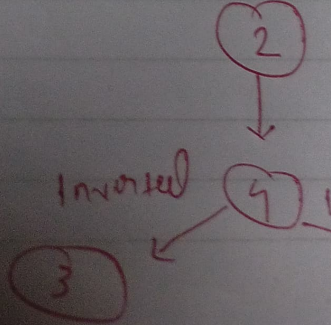
Ex

Given, g_nodes = 4 , g_from = [1, 2, 3] and g_to = [4, 4, 4]

tree can be visualize as:



→ One of the optimal strategies is to select node 2 as the root and invert the edges 1 to 4 and 3 to 4 thus the minimum number of edges to be inverted is 2.



Hence the ans is 2

DATE / /
PAGE

⇒ Function

getMinInversions (int g_nodes, vector<int> g_from, vector<int> g_to)

- g_nodes: # node in G.
- g_from: starting end of the directed edges
- g_to: the ending node of directed edges

⇒ Return

int: the minimum num of edges that that should be inverted.

⇒ Constraints

$2 \leq g\_nodes \leq 10^5$

$1 \leq g\_from[i], g\_to[i] \leq g\_nodes$

$g\_from[i] \neq g\_to[i]$

⇒ Input format

the first line contain two space seperated integers, g_node, and g_edges.

the following g_edges lines contain two space-seperated integer each, g_from[i] and g_to[i] respectively.

⇒ Sample Case 0.

STDIN FUNCTION

3 2 → g_nodes = 3, g_edges = 2

2 1 → g_from = [2,2), g_to = 2, 3

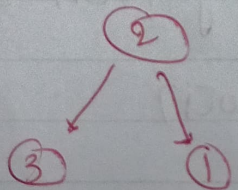
[1, 3]

2/3

Sample o/p

0

The optimal strategy is to root the tree at node 2



⇒ Sample Case-1

STDIN FUNCTION

4 3 → n = 4, g_edges = 3

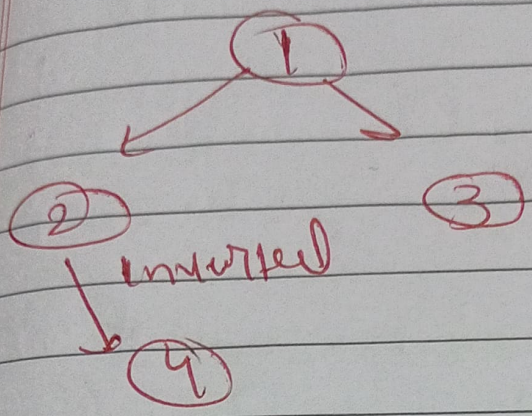
1 3 → g_from = [1,1,4], g_to [3,3,2]

~~[3, 2, 3]~~ 1, 2

4 2

DATE / /
PAGE

Sample o/p [1]



55m left

ALL

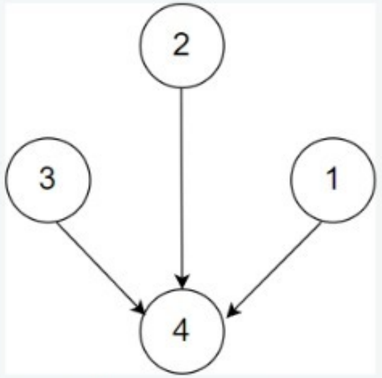
1

2

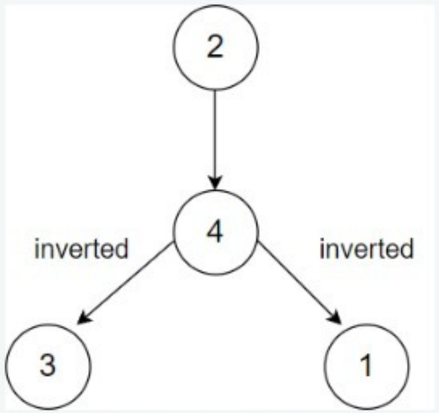
3

Example

Given, *g_nodes* = 4, *g_from* = *[1, 2, 3]*, and *g_to* = *[4, 4, 4]*. The tree can be visualized as:



One of the optimal strategies is to select node 2 as the root and invert the edges 1 to 4 and 3 to 4. Thus the minimum number of edges to be inverted is 2.



Hence, the answer is 2.

Function Description

Complete the function *getMinInversions* in the editor below.

Language C++14 | Autocomplete Ready | Environment

```
1  #include <bits/stdc++.h>
10
11 /*
12  * Complete the 'getMinInversions' function below.
13  *
14  * The function is expected to return an INTEGER.
15  * The function accepts UNWEIGHTED_INTEGER_GRAPH g as parameter.
16  */
17
18 /*
19  * For the unweighted graph, <name>:
20  *
21  * 1. The number of nodes is <name>_nodes.
22  * 2. The number of edges is <name>_edges.
23  * 3. An edge exists between <name>_from[i] and <name>_to[i].
24  *
25  */
26
27 int getMinInversions(int g_nodes, vector<int> g_from, vector<int> g_to) {
28
29 }
30
31 int main()
```

Line: 31 Col: 11

Test Results | Custom Input | Run Code | Run Tests | Submit

55m left

ALL

1

2

3

Returns

int: the minimum number of edges that must be inverted

Constraints

- $2 \le g\_nodes \le 10^5$
- $1 \le g\_from[i], g\_to[i] \le g\_nodes$
- $g\_from[i] \ne g\_to[i]$

▸ **Input Format For Custom Testing**

▾ **Sample Case 0**

Sample Input For Custom Testing

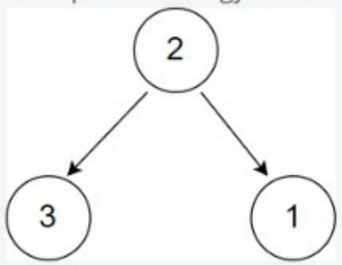
```
STDIN           FUNCTION
-----           --------
3 2       →     g_nodes = 3, g_edges = 2
2 1       →     g_from = [2, 2], g_to = [1, 3]
2 3
```

Sample Output

```
0
```

Explanation

The optimal strategy is to root the tree at node 2.



Language C++14 | Autocomplete Ready | Environment

```
1  #include <bits/stdc++.h> ···
10
11 /*
12  * Complete the 'getMinInversions' function below.
13  *
14  * The function is expected to return an INTEGER.
15  * The function accepts UNWEIGHTED_INTEGER_GRAPH g as parameter.
16  */
17
18 /*
19  * For the unweighted graph, <name>:
20  *
21  * 1. The number of nodes is <name>_nodes.
22  * 2. The number of edges is <name>_edges.
23  * 3. An edge exists between <name>_from[i] and <name>_to[i].
24  *
25  */
26
27 int getMinInversions(int g_nodes, vector<int> g_from, vector<int> g_to) {
28
29 }
30
31 int main() ···
```

Line: 31 Col: 11

Test Results | Custom Input | Run Code | Run Tests | Submit

55m left

ALL

1

2

3

2. Question 2

A ship has *n* doors locked and the captain calls out an integer *k*. For the crew to get on the ship, all doors must be unlocked. The array *locks* represents *n* locks, where the i^{th} lock has an initial value *locks[i]*. All the doors will unlock only when the values on all locks are the same.

In one operation, consider the current value of the i^{th} lock.

1. If the current value = 1, its next value can be *k* or 2
2. If the current value = *k*, its next value can be *k - 1* or 1
3. Otherwise, its next value can be *current value - 1* or *current value + 1*.

Find the minimum number of operations such that the values on all locks are the same.

For example, *k = 100, n = 3, locks = [1, 2, 99]*.

In the optimal assignment, the final common value should be 1

1. For 1 - operations required = 0
2. For 2 - operations required = 1 i.e 2 -> 1
3. For 99 - operations required = 2 i.e 99 -> 100 -> 1

It takes 0 + 1 + 2 = 3 operations to make all the values equal. Therefore, the minimum number of operations required is equal to 3.

Language C++14 | Environment | Autocomplete Ready

```
#include <bits/stdc++.h>···

/*
 * Complete the 'minOperations' function below.
 *
 * The function is expected to return a LONG_INTEGER.
 * The function accepts following parameters:
 *  1. INTEGER k
 *  2. INTEGER_ARRAY locks
 */

long minOperations(int k, vector<int> locks) {

}

int main()···
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

Search

ENG IN 19:09 01-08-2024

55m left

ALL

1

2

3

It takes 0 + 1 + 2 = 3 operations to make all the values equal. Therefore, the minimum number of operations required is equal to 3.

Function Description

Complete the function *minOperations* in the editor below.

minOperations has the following parameter(s):

int k: an integer

int locks[n]: the initial lock values

Returns

long int: the minimum number of operations required to have a common value on all locks

Constraints

- $2 \le k \le 10^9$
- $1 \le n \le 10^5$
- $1 \le locks[i] \le k$

▸ **Input Format For Custom Testing**

▾ **Sample Case 0**

Sample Input For Custom Testing

```
100
3
1
2
98
```

Language C++14 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h>...
9
10 /*
11  * Complete the 'minOperations' function below.
12  *
13  * The function is expected to return a LONG_INTEGER.
14  * The function accepts following parameters:
15  *  1. INTEGER k
16  *  2. INTEGER_ARRAY locks
17  */
18
19 long minOperations(int k, vector<int> locks) {
20
21 }
22
23 int main()...
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

55m left

ALL

1

2

3

Returns

long int: the minimum number of operations required to have a common value on all locks

Constraints

- $2 \leq k \leq 10^9$
- $1 \leq n \leq 10^5$
- $1 \leq locks[i] \leq k$

▸ **Input Format For Custom Testing**

▾ **Sample Case 0**

Sample Input For Custom Testing

```
100
3
1
2
98
```

Sample Output

```
4
```

Explanation

Here, k = 100, a = [1, 2, 98]. In the optimal configuration, the common value is 1. Therefore, operations required = 0 + 1 + (2 + 1) = 4

▸ **Sample Case 1**

Language C++14 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h>...
9
10 /*
11  * Complete the 'minOperations' function below.
12  *
13  * The function is expected to return a LONG_INTEGER.
14  * The function accepts following parameters:
15  *  1. INTEGER k
16  *  2. INTEGER_ARRAY locks
17  */
18
19 long minOperations(int k, vector<int> locks) {
20
21 }
22
23 int main()...
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

54m left

ALL

1

2

3

3. Question 3

There is network of radio-wave devices which can transmit waves of integer frequencies of 1, 2 or 3 units. Two devices can receive messages directly if their transmission frequencies have an absolute difference of 1, at most. The network is given by a tree having *network_nodes* number of devices and *network_nodes - 1* number of edges. The edges are numbered from *1* to *network_nodes*, in the form of two arrays, *network_from*, and *network_to* respectively. There is an undirected edge from *network_from[i]* to *network_to[i]* *(*1 ≤ *i* < *network_nodes)*. There is also an array of frequency values, *frequency*, of size *network_nodes*.

Determine the longest distance between two devices that can transmit their message to each other. A device can transmit a message to another node if there is a simple path such that compatibility values of two consecutive devices differ by no more than one. If no device can transmit a message, return 0.

Note: The distance between two devices is defined as the number of edges in a simple path from one node to the other. A simple path is a sequence of devices from one node to another such that each consecutive device is connected by an edge and no node is used more than once in the sequence

Language C++14 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h>···
10
11 /*
12  * Complete the 'calculateMax' function below.
13  *
14  * The function is expected to return an INTEGER.
15  * The function accepts following parameters:
16  *  1. UNWEIGHTED_INTEGER_GRAPH network
17  *  2. INTEGER_ARRAY frequency
18  */
19
20 /*
21  * For the unweighted graph, <name>:
22  *
23  * 1. The number of nodes is <name>_nodes.
24  * 2. The number of edges is <name>_edges.
25  * 3. An edge exists between <name>_from[i] and <name>_to[i].
26  *
27  */
28
29 int calculateMax(int network_nodes, vector<int> network_from, vector<int> network_to, vector<int>
   frequency) {
30
31 }
32
33 int main()···
```

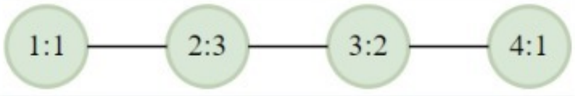
Line: 10 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

node is used more than once in the sequence.

Example

network_nodes = 4, *network_from* = [1, 2, 3], *network_to* = [2, 3, 4], and *frequency* = [1, 3, 2, 1]



The graph is labeled *device:frequency*. Check if a message can be passed for each pair of devices and distance.

- (1, 2) : The difference between frequencies, 1 and 3, is greater than 1. A message cannot be transmitted.
- (1, 3) : A message cannot be transmitted due to failure at (1, 2).
- (1, 4) : A message cannot be transmitted due to failure at (1, 2).
- (2, 3) : The difference in frequencies, 3 and 2, is 1. A message can be transmitted.
- (2, 4) : Differences are such that a message can be transmitted from 2->3 and 3->4. The distance is 2.
- (3, 4) : A message can be transmitted, and the distance is 1.

The longest path, and the answer, is 2.

Function Description

Complete the function *calculateMax* in the editor below.

calculateMax has the following parameter(s):

Language C++14 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h> ···
10
11 /*
12  * Complete the 'calculateMax' function below.
13  *
14  * The function is expected to return an INTEGER.
15  * The function accepts following parameters:
16  *  1. UNWEIGHTED_INTEGER_GRAPH network
17  *  2. INTEGER_ARRAY frequency
18  */
19
20 /*
21  * For the unweighted graph, <name>:
22  *
23  * 1. The number of nodes is <name>_nodes.
24  * 2. The number of edges is <name>_edges.
25  * 3. An edge exists between <name>_from[i] and <name>_to[i].
26  *
27  */
28
29 int calculateMax(int network_nodes, vector<int> network_from, vector<int> network_to, vector<int>
    frequency) {
30
31 }
32
33 int main() ···
```

Line: 10 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

54m left

ALL

1

2

3

Function Description
Complete the function *calculateMax* in the editor below.

calculateMax has the following parameter(s):
int network_nodes: the number of devices
int network_from[network_nodes - 1]: one end of the edge
int network_to[network_nodes - 1]: the other end of the edge
int frequency[network_nodes]: compatibility values for each node.

Returns
int: the longest distance between any pair of devices that can transmit messages to each other

Constraints

- $2 \le network\_nodes \le 2 * 10^5$
- $1 \le network\_from[i], network\_to[i] \le network\_nodes$
- $1 \le frequency[i] \le 3$

▸ **Input Format For Custom Testing**

▾ **Sample Case 0**

Sample Input For Custom Testing

STDIN	FUNCTION
-----	--------

Language C++14 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h> ···
10
11 /*
12  * Complete the 'calculateMax' function below.
13  *
14  * The function is expected to return an INTEGER.
15  * The function accepts following parameters:
16  *  1. UNWEIGHTED_INTEGER_GRAPH network
17  *  2. INTEGER_ARRAY frequency
18  */
19
20 /*
21  * For the unweighted graph, <name>:
22  *
23  * 1. The number of nodes is <name>_nodes.
24  * 2. The number of edges is <name>_edges.
25  * 3. An edge exists between <name>_from[i] and <name>_to[i].
26  *
27  */
28
29 int calculateMax(int network_nodes, vector<int> network_from, vector<int> network_to, vector<int>
   frequency) {
30
31 }
32
33 int main() ···
```

Line: 10 Col: 1

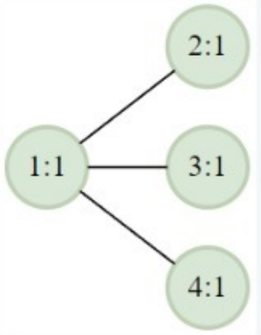
Test Results | Custom Input | Run Code | Run Tests | Submit

4, network_nodes - 1 = 3
1 2 → network_from = [1, 1, 1], network_to = [2, 3, 4]
1 3
1 4
4 → frequency[] size network_nodes = 4
1 → frequency = [1, 1, 1, 1]
1
1
1

Sample Output

2

Explanation



Messages can pass between all devices since they have the same frequency, i.e. their difference is 0. The longest path is 2, for example from device 2 to device 4.

▸ **Sample Case 1**

Language C++14 | Environment | Autocomplete Ready

```
#include <bits/stdc++.h> ···

/*
 * Complete the 'calculateMax' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts following parameters:
 *  1. UNWEIGHTED_INTEGER_GRAPH network
 *  2. INTEGER_ARRAY frequency
 */

/*
 * For the unweighted graph, <name>:
 *
 * 1. The number of nodes is <name>_nodes.
 * 2. The number of edges is <name>_edges.
 * 3. An edge exists between <name>_from[i] and <name>_to[i].
 *
 */

int calculateMax(int network_nodes, vector<int> network_from, vector<int> network_to, vector<int> frequency) {

}

int main() ···
```

Line: 10 Col: 1

Test Results **Custom Input** | **Run Code** | **Run Tests** | **Submit**